

Data-driven Model Discovery and Neural Operators

Xiaofeng Xu

Postdoctoral Scholar,
Department of Mathematics, Penn State University

OneMath World School
Brin Mathematical Research Center
University of Maryland
June 18, 2026

- 1 Background and Standard Operator Learning
- 2 Problem Statement and Challenge
- 3 Laplacian-Eigenfunction Neural Operator (LENO)-based method
- 4 Numerical examples

Standard Operator Learning Setup

- Input function: $v : D \subset \mathbb{R}^d \rightarrow \mathbb{R}$
- Output function: $u : D' \subset \mathbb{R}^d \rightarrow \mathbb{R}$
- Operator mapping: $\mathcal{G} : \mathcal{V} \rightarrow \mathcal{U}, \quad v \mapsto u$

Standard Operator Learning Setup

- Input function: $v : D \subset \mathbb{R}^d \rightarrow \mathbb{R}$
- Output function: $u : D' \subset \mathbb{R}^d \rightarrow \mathbb{R}$
- Operator mapping: $\mathcal{G} : \mathcal{V} \rightarrow \mathcal{U}, \quad v \mapsto u$
- **Goal:** Learn $\mathcal{G}$ using neural networks with training data

Standard Operator Learning Setup

- Input function: $v : D \subset \mathbb{R}^d \rightarrow \mathbb{R}$
- Output function: $u : D' \subset \mathbb{R}^d \rightarrow \mathbb{R}$
- Operator mapping: $\mathcal{G} : \mathcal{V} \rightarrow \mathcal{U}, \quad v \mapsto u$
- **Goal:** Learn $\mathcal{G}$ using neural networks with training data

$$\mathcal{T} = \{(v^{(i)}, u^{(i)})\}_{i=1}^m$$

Standard Operator Learning Setup

- Input function: $v : D \subset \mathbb{R}^d \rightarrow \mathbb{R}$
- Output function: $u : D' \subset \mathbb{R}^d \rightarrow \mathbb{R}$
- Operator mapping: $\mathcal{G} : \mathcal{V} \rightarrow \mathcal{U}, \quad v \mapsto u$
- **Goal:** Learn $\mathcal{G}$ using neural networks with training data

$$\mathcal{T} = \{(v^{(i)}, u^{(i)})\}_{i=1}^m$$

Applications:

Standard Operator Learning Setup

- Input function: $v : D \subset \mathbb{R}^d \rightarrow \mathbb{R}$
- Output function: $u : D' \subset \mathbb{R}^d \rightarrow \mathbb{R}$
- Operator mapping: $\mathcal{G} : \mathcal{V} \rightarrow \mathcal{U}, \quad v \mapsto u$
- **Goal:** Learn $\mathcal{G}$ using neural networks with training data

$$\mathcal{T} = \{(v^{(i)}, u^{(i)})\}_{i=1}^m$$

Applications:

- PDE solution operator
- Parameter-to-solve map
- ...

DeepONet Architecture

- DeepONet $\mathcal{N}$ approximates an operator $\mathcal{G} : v \mapsto u$:

DeepONet Architecture

- DeepONet $\mathcal{N}$ approximates an operator $\mathcal{G} : v \mapsto u$:
 - ▶ Trunk network: takes the coordinate $x \in \mathbb{R}^d$ and outputs basis functions $\mathbf{t}(x) \in \mathbb{R}^p = [t_1(x), \dots, t_p(x)]^T$.

DeepONet Architecture

- DeepONet $\mathcal{N}$ approximates an operator $\mathcal{G} : v \mapsto u$:
 - ▶ Trunk network: takes the coordinate $x \in \mathbb{R}^d$ and outputs basis functions $\mathbf{t}(x) \in \mathbb{R}^p = [t_1(x), \dots, t_p(x)]^T$.
 - ▶ Branch network: takes discretized input function $\mathbf{v} := [v(x_1), \dots, v(x_m)] \in \mathbb{R}^m$ and outputs coefficients $\mathbf{b}(\mathbf{v}) \in \mathbb{R}^p = [b_1(\mathbf{v}), \dots, b_p(\mathbf{v})]^T$.

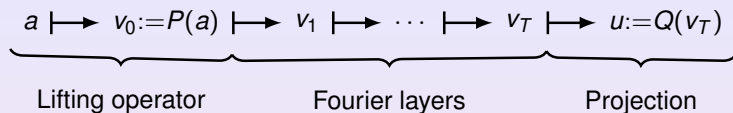
DeepONet Architecture

- DeepONet $\mathcal{N}$ approximates an operator $\mathcal{G} : v \mapsto u$:
 - ▶ Trunk network: takes the coordinate $x \in \mathbb{R}^d$ and outputs basis functions $\mathbf{t}(x) \in \mathbb{R}^p = [t_1(x), \dots, t_p(x)]^T$.
 - ▶ Branch network: takes discretized input function $\mathbf{v} := [v(x_1), \dots, v(x_m)] \in \mathbb{R}^m$ and outputs coefficients $\mathbf{b}(\mathbf{v}) \in \mathbb{R}^p = [b_1(\mathbf{v}), \dots, b_p(\mathbf{v})]^T$.
- DeepONet architecture:

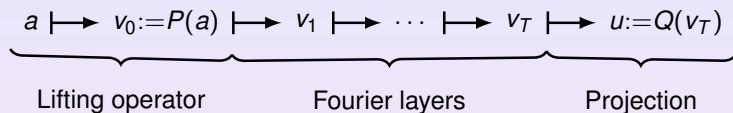
$$\mathcal{N}(v; \theta)(x) = \mathbf{b}(\mathbf{v}; \theta_1) \cdot \mathbf{t}(x; \theta_2) := \sum_{k=1}^p b_k t_k.$$

- ▶ Trunk net $\mathbf{t}(x; \theta_2)$: typically a fully connected neural network (FNN).
- ▶ Branch net $\mathbf{b}(v; \theta_1)$: FNN, CNN, RNN, ResNet, etc.

Fourier Neural Operator (FNO)

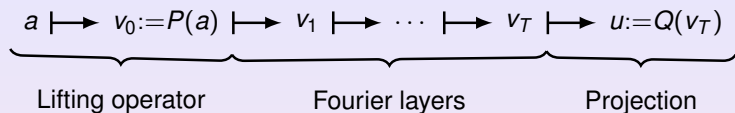


Fourier Neural Operator (FNO)



- The lifting operator P maps the input function $a(x)$ to a higher-dimensional latent representation
- Q projects the latent representation back to the output space.

Fourier Neural Operator (FNO)



- The lifting operator P maps the input function $a(x)$ to a higher-dimensional latent representation
- Q projects the latent representation back to the output space.
- The latent features are updated through

$$v_{t+1}(x) := \sigma((\mathcal{W}v_t)(x) + (\mathcal{K}_\theta v_t)(x)).$$

- $\mathcal{W}$ is a linear operator, and $\mathcal{K}_\theta$ is an integral operator

$$(\mathcal{K}_\theta v_t)(x) = \int_D \kappa_\theta(x, y) v_t(y) dy.$$

Fourier Neural Operator (FNO)

- FNO: translation-invariant kernel $\kappa_{\theta}(x, y) = \kappa_{\theta}(x - y)$

Fourier Neural Operator (FNO)

- FNO: translation-invariant kernel $\kappa_\theta(x, y) = \kappa_\theta(x - y)$
- By the convolution theorem, we have

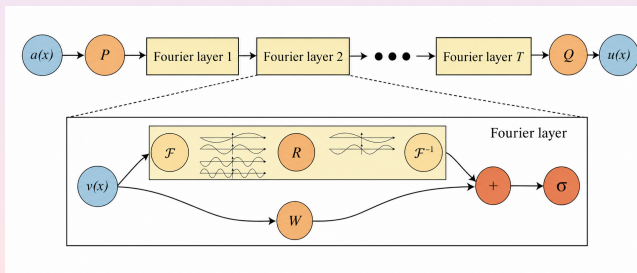
$$(\mathcal{K}_\theta v_t)(x) = \kappa_\theta * v_t(x) = \mathcal{F}^{-1}(\mathcal{F}(\kappa_\theta) \cdot \mathcal{F}(v_t)) \quad (1)$$

Fourier Neural Operator (FNO)

- FNO: translation-invariant kernel $\kappa_{\theta}(x, y) = \kappa_{\theta}(x - y)$
- By the convolution theorem, we have

$$(\mathcal{K}_{\theta} v_t)(x) = \kappa_{\theta} * v_t(x) = \mathcal{F}^{-1} (\mathcal{F}(\kappa_{\theta}) \cdot \mathcal{F}(v_t)) \quad (1)$$

- Fourier integral operator: $\mathcal{K}_{\theta} v_t(x) = \mathcal{F}^{-1} (R_{\theta} \cdot \mathcal{F}(v_t))$



An Example: Darcy Flow

$$\begin{aligned} -\nabla \cdot (a(x) \nabla u(x)) &= f(x), & x &\in (0, 1)^2, \\ u(x) &= 0, & x &\in \partial(0, 1)^2. \end{aligned}$$

An Example: Darcy Flow

$$\begin{aligned} -\nabla \cdot (a(x) \nabla u(x)) &= f(x), & x \in (0, 1)^2, \\ u(x) &= 0, & x \in \partial(0, 1)^2. \end{aligned}$$

- **Solution operator learning:**

Learn $\mathcal{G} : f(x) \mapsto u(x)$

Training pairs (f_i, u_i) are available:

$\{(f_i, u_i)\}_{i=1}^N \implies$ Train DeepONet or FNO to approximate $\mathcal{G}$.

An Example: Darcy Flow

$$\begin{aligned} -\nabla \cdot (a(x) \nabla u(x)) &= f(x), & x \in (0, 1)^2, \\ u(x) &= 0, & x \in \partial(0, 1)^2. \end{aligned}$$

- **Solution operator learning:**

$$\text{Learn } \mathcal{G} : f(x) \mapsto u(x)$$

Training pairs (f_i, u_i) are available:

$$\{(f_i, u_i)\}_{i=1}^N \implies \text{Train DeepONet or FNO to approximate } \mathcal{G}.$$

- **Parameter-to-solution map learning:**

$$\text{Learn } \mathcal{G} : a(x) \mapsto u(x).$$

Training pairs (a_i, u_i) are available:

$$\{(a_i, u_i)\}_{i=1}^N \implies \text{Train DeepONet or FNO to approximate } \mathcal{G}.$$

- Code is available at <https://github.com/georgexxu/>

Data-driven-Model-Discovery-Neural-Operators.

- 1 Background and Standard Operator Learning
- 2 Problem Statement and Challenge
- 3 Laplacian-Eigenfunction Neural Operator (LENO)-based method
- 4 Numerical examples

Standard Operator Learning vs Our Problem

- **Standard operator learning**

Training pairs $(v^{(i)}, u^{(i)})$ are available:

Learn $\mathcal{G} : v \mapsto u$

Standard Operator Learning vs Our Problem

- **Standard operator learning**

Training pairs $(v^{(i)}, u^{(i)})$ are available:

$$\text{Learn } \mathcal{G} : v \mapsto u$$

- **Our model problem**

Reaction-diffusion equation:

$$u_t - \Delta u = \mathcal{F}(u).$$

Standard Operator Learning vs Our Problem

- **Standard operator learning**

Training pairs $(v^{(i)}, u^{(i)})$ are available:

$$\text{Learn } \mathcal{G} : v \mapsto u$$

- **Our model problem**

Reaction-diffusion equation:

$$u_t - \Delta u = \mathcal{F}(u).$$

$\mathcal{F}(u)$ is unknown.

Standard Operator Learning vs Our Problem

- **Standard operator learning**

Training pairs $(v^{(i)}, u^{(i)})$ are available:

$$\text{Learn } \mathcal{G} : v \mapsto u$$

- **Our model problem**

Reaction-diffusion equation:

$$u_t - \Delta u = \mathcal{F}(u).$$

$\mathcal{F}(u)$ is unknown.

Observations: $\{(t_i, u^i)\}_{i=1}^N$

Standard Operator Learning vs Our Problem

- **Standard operator learning**

Training pairs $(v^{(i)}, u^{(i)})$ are available:

$$\text{Learn } \mathcal{G} : v \mapsto u$$

- **Our model problem**

Reaction-diffusion equation:

$$u_t - \Delta u = \mathcal{F}(u).$$

$\mathcal{F}(u)$ is unknown.

Observations: $\{(t_i, u^i)\}_{i=1}^N$

Direct training pairs $(u^i, \mathcal{F}(u^i))$ are not available.

Standard Operator Learning vs Our Problem

- **Standard operator learning**

Training pairs $(v^{(i)}, u^{(i)})$ are available:

$$\text{Learn } \mathcal{G} : v \mapsto u$$

- **Our model problem**

Reaction-diffusion equation:

$$u_t - \Delta u = \mathcal{F}(u).$$

$\mathcal{F}(u)$ is unknown.

Observations: $\{(t_i, u^i)\}_{i=1}^N$

Direct training pairs $(u^i, \mathcal{F}(u^i))$ are not available.

How to learn $\mathcal{F}(u)$?

Learn the Nonlinear Reaction Operator

Consider the reaction–diffusion equation

$$u_t - \Delta u = \mathcal{F}(u),$$

with only observations $\{(t_n, u^n(x))\}_{n=0}^N$.

Learn the unknown operator $\mathcal{F}(u) \approx \mathcal{N}(u; \theta)$.

Learn the Nonlinear Reaction Operator

Consider the reaction–diffusion equation

$$u_t - \Delta u = \mathcal{F}(u),$$

with only observations $\{(t_n, u^n(x))\}_{n=0}^N$.

Learn the unknown operator $\mathcal{F}(u) \approx \mathcal{N}(u; \theta)$.

Idea:

Learn the Nonlinear Reaction Operator

Consider the reaction–diffusion equation

$$u_t - \Delta u = \mathcal{F}(u),$$

with only observations $\{(t_n, u^n(x))\}_{n=0}^N$.

Learn the unknown operator $\mathcal{F}(u) \approx \mathcal{N}(u; \theta)$.

Idea:

- Construct surrogate labels from the discretized PDE:

$$\mathcal{F}(u^n) \approx \frac{u^{n+1} - u^n}{t_{n+1} - t_n} - \Delta u^{n+1} =: R(u^n).$$

Learn the Nonlinear Reaction Operator

Consider the reaction–diffusion equation

$$u_t - \Delta u = \mathcal{F}(u),$$

with only observations $\{(t_n, u^n(x))\}_{n=0}^N$.

Learn the unknown operator $\mathcal{F}(u) \approx \mathcal{N}(u; \theta)$.

Idea:

- Construct surrogate labels from the discretized PDE:

$$\mathcal{F}(u^n) \approx \frac{u^{n+1} - u^n}{t_{n+1} - t_n} - \Delta u^{n+1} =: R(u^n).$$

- Evolve the PDE with the nonlinear term $\mathcal{N}(u; \theta)$:

$$\frac{\tilde{u}^{n+1} - \tilde{u}^n}{t_{n+1} - t_n} - \Delta \tilde{u}^{n+1} = \mathcal{N}(\tilde{u}^n; \theta).$$

- 1 Background and Standard Operator Learning
- 2 Problem Statement and Challenge
- 3 Laplacian-Eigenfunction Neural Operator (LENO)-based method**
- 4 Numerical examples

Laplacian-Eigenfunction Neural Operator (LENO)-based method



$$\begin{cases} u_t - \Delta u = \mathcal{F}(u), & \text{in } \Omega \times (0, T) \\ u(x, 0) = u_0(x), & \text{in } \Omega \end{cases} \quad (2)$$

$\mathcal{F}(u)$ is unknown. Observations $\{(t_n, u^n(x))\}_{n=0}^N$.

Laplacian-Eigenfunction Neural Operator (LENO)-based method



$$\begin{cases} u_t - \Delta u = \mathcal{F}(u), & \text{in } \Omega \times (0, T) \\ u(x, 0) = u_0(x), & \text{in } \Omega \end{cases} \quad (2)$$

$\mathcal{F}(u)$ is unknown. Observations $\{(t_n, u^n(x))\}_{n=0}^N$.

- Truncated eigenspace:

$$V_P = \text{span}\{\phi_i\}_{i=1}^P$$

Laplacian-Eigenfunction Neural Operator (LENO)-based method



$$\begin{cases} u_t - \Delta u = \mathcal{F}(u), & \text{in } \Omega \times (0, T) \\ u(x, 0) = u_0(x), & \text{in } \Omega \end{cases} \quad (2)$$

$\mathcal{F}(u)$ is unknown. Observations $\{(t_n, u^n(x))\}_{n=0}^N$.

- Truncated eigenspace:

$$V_P = \text{span}\{\phi_i\}_{i=1}^P$$

ϕ_i are the eigenfunctions satisfying $-\Delta\phi_i = \lambda_i\phi_i$.

Laplacian-Eigenfunction Neural Operator (LENO)-based method



$$\begin{cases} u_t - \Delta u = \mathcal{F}(u), & \text{in } \Omega \times (0, T) \\ u(x, 0) = u_0(x), & \text{in } \Omega \end{cases} \quad (2)$$

$\mathcal{F}(u)$ is unknown. Observations $\{(t_n, u^n(x))\}_{n=0}^N$.

- Truncated eigenspace:

$$V_P = \text{span}\{\phi_i\}_{i=1}^P$$

ϕ_i are the eigenfunctions satisfying $-\Delta\phi_i = \lambda_i\phi_i$.

- The LENO architecture is given by

$$\mathcal{N}(u; \theta)(x) = \sum_{i=1}^P b_k t_k = \sum_{i=1}^P \mathcal{G}_i(\beta(u); \theta) \phi_i(x) \approx \mathcal{F}(u).$$

Laplacian-Eigenfunction Neural Operator (LENO)-based method



$$\begin{cases} u_t - \Delta u = \mathcal{F}(u), & \text{in } \Omega \times (0, T) \\ u(x, 0) = u_0(x), & \text{in } \Omega \end{cases} \quad (2)$$

$\mathcal{F}(u)$ is unknown. Observations $\{(t_n, u^n(x))\}_{n=0}^N$.

- Truncated eigenspace:

$$V_P = \text{span}\{\phi_i\}_{i=1}^P$$

ϕ_i are the eigenfunctions satisfying $-\Delta\phi_i = \lambda_i\phi_i$.

- The LENO architecture is given by

$$\mathcal{N}(u; \theta)(x) = \sum_{i=1}^P b_k t_k = \sum_{i=1}^P \mathcal{G}_i(\beta(u); \theta) \phi_i(x) \approx \mathcal{F}(u).$$

- ▶ $\beta(u) \in \mathbb{R}^P$ is the projection of u onto the eigenspace V_P .

Laplacian-Eigenfunction Neural Operator (LENO)-based method



$$\begin{cases} u_t - \Delta u = \mathcal{F}(u), & \text{in } \Omega \times (0, T) \\ u(x, 0) = u_0(x), & \text{in } \Omega \end{cases} \quad (2)$$

$\mathcal{F}(u)$ is unknown. Observations $\{(t_n, u^n(x))\}_{n=0}^N$.

- Truncated eigenspace:

$$V_P = \text{span}\{\phi_i\}_{i=1}^P$$

ϕ_i are the eigenfunctions satisfying $-\Delta\phi_i = \lambda_i\phi_i$.

- The LENO architecture is given by

$$\mathcal{N}(u; \theta)(x) = \sum_{i=1}^P b_k t_k = \sum_{i=1}^P \mathcal{G}_i(\beta(u); \theta) \phi_i(x) \approx \mathcal{F}(u).$$

- ▶ $\beta(u) \in \mathbb{R}^P$ is the projection of u onto the eigenspace V_P .
- ▶ The branch network : a fully connected neural network

LENO: solve the equation in the eigenspace

LENO: solve the equation in the eigenspace

- Data projection: $\beta_i^n(u) := (u^n, \phi_i)_{L^2}$

LENO: solve the equation in the eigenspace

- Data projection: $\beta_i^n(u) := (u^n, \phi_i)_{L^2}$
- Equation projection: $\tilde{u}^n(x) := \sum_{i=1}^P \tilde{\beta}_i^n \phi_i(x)$.

LENO: solve the equation in the eigenspace

- Data projection: $\beta_i^n(u) := (u^n, \phi_i)_{L^2}$
- Equation projection: $\tilde{u}^n(x) := \sum_{i=1}^P \tilde{\beta}_i^n \phi_i(x)$.
- Applying a semi-implicit scheme

$$\frac{\tilde{\beta}^n - \tilde{\beta}^{n-1}}{t_n - t_{n-1}} + \Lambda_P \tilde{\beta}^n = \tilde{f}^{n-1}, \quad (3)$$

where $\tilde{f}_i^{n-1} = (\mathcal{F}(\tilde{u}^{n-1}), \phi_i)$, $\Lambda_P = \text{diag}(\lambda_1, \dots, \lambda_P)$.

LENO: solve the equation in the eigenspace

- Data projection: $\beta_i^n(u) := (u^n, \phi_i)_{L^2}$
- Equation projection: $\tilde{u}^n(x) := \sum_{i=1}^P \tilde{\beta}_i^n \phi_i(x)$.
- Applying a semi-implicit scheme

$$\frac{\tilde{\beta}^n - \tilde{\beta}^{n-1}}{t_n - t_{n-1}} + \Lambda_P \tilde{\beta}^n = \tilde{f}^{n-1}, \quad (3)$$

where $\tilde{f}_i^{n-1} = (\mathcal{F}(\tilde{u}^{n-1}), \phi_i)$, $\Lambda_P = \text{diag}(\lambda_1, \dots, \lambda_P)$.

- LENO parametrization: $\sum_{i=1}^P \mathcal{G}_i(\beta(u); \theta) \phi_i(x) \approx \mathcal{F}(u)$:

$$\frac{\tilde{\beta}^n - \tilde{\beta}^{n-1}}{t_n - t_{n-1}} + \Lambda_P \tilde{\beta}^n = \mathcal{G}(\tilde{\beta}^{n-1}; \theta), \quad (4)$$

where $\mathcal{G}(\tilde{\beta}^{n-1}; \theta)_i := \mathcal{G}_i(\tilde{\beta}^{n-1}; \theta)$.

- Loss function formulation: minimizing the data mismatch and the residual of the discretized PDE.

Loss function formulation

Loss function formulation

- Define

$$\mathcal{R}^n(u) := \frac{\beta^n(u) - \beta^{n-1}(u)}{t_n - t_{n-1}} + \Lambda_P \beta^n(u). \quad (5)$$

Loss function formulation

- Define

$$\mathcal{R}^n(u) := \frac{\beta^n(u) - \beta^{n-1}(u)}{t_n - t_{n-1}} + \Lambda_P \beta^n(u). \quad (5)$$

- Model residual loss ensures that the solution satisfies the model problem

$$L^R(\theta) := \frac{1}{N} \sum_{n=1}^N \frac{\|\mathcal{R}^n - \mathcal{G}(\beta^{n-1}; \theta)\|_{\ell^2}}{\|\mathcal{R}^n\|_{\ell^2}}. \quad (6)$$

Loss function formulation

- Define

$$\mathcal{R}^n(u) := \frac{\beta^n(u) - \beta^{n-1}(u)}{t_n - t_{n-1}} + \Lambda_P \beta^n(u). \quad (5)$$

- Model residual loss ensures that the solution satisfies the model problem

$$L^R(\theta) := \frac{1}{N} \sum_{n=1}^N \frac{\|\mathcal{R}^n - \mathcal{G}(\beta^{n-1}; \theta)\|_{\ell^2}}{\|\mathcal{R}^n\|_{\ell^2}}. \quad (6)$$

- Data L^2 loss ensures that the discretized solution $\tilde{\beta}^n$ closely matches the given data β^n and is given by

$$L^D(\theta) := \frac{1}{N} \sum_{n=1}^N \frac{\|\tilde{\beta}^n - \beta^n\|_{\ell^2}}{\|\beta^n\|_{\ell^2}}. \quad (7)$$

Loss function formulation

- Define

$$\mathcal{R}^n(u) := \frac{\beta^n(u) - \beta^{n-1}(u)}{t_n - t_{n-1}} + \Lambda_P \beta^n(u). \quad (5)$$

- Model residual loss ensures that the solution satisfies the model problem

$$L^R(\theta) := \frac{1}{N} \sum_{n=1}^N \frac{\|\mathcal{R}^n - \mathcal{G}(\beta^{n-1}; \theta)\|_{\ell^2}}{\|\mathcal{R}^n\|_{\ell^2}}. \quad (6)$$

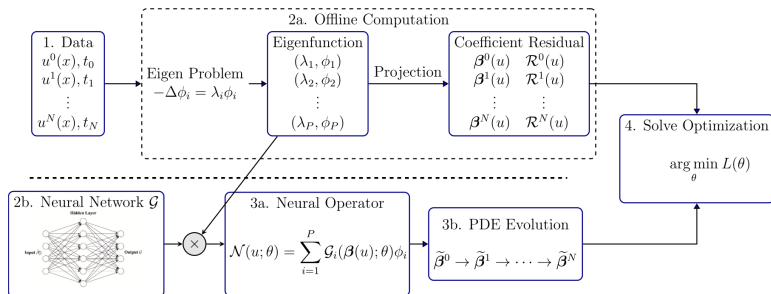
- Data L^2 loss ensures that the discretized solution $\tilde{\beta}^n$ closely matches the given data β^n and is given by

$$L^D(\theta) := \frac{1}{N} \sum_{n=1}^N \frac{\|\tilde{\beta}^n - \beta^n\|_{\ell^2}}{\|\beta^n\|_{\ell^2}}. \quad (7)$$

- The neural operator is then trained by minimizing the following loss function

$$L(\theta) := L^D(\theta) + L^R(\theta). \quad (8)$$

LENO workflow for learning the nonlinear reaction term



- 1 Background and Standard Operator Learning
- 2 Problem Statement and Challenge
- 3 Laplacian-Eigenfunction Neural Operator (LENO)-based method
- 4 Numerical examples**

Example 1. KPP-Fisher equation

- The KPP-Fisher equation is a nonlinear reaction-diffusion equation that models the propagation of a population.

$$\begin{aligned}\frac{\partial u}{\partial t} - \frac{\partial^2 u}{\partial x^2} &= u(1 - u), \quad t \in (0, T], \\ u(x, 0) &= u_0(x), \quad x \in (0, 1),\end{aligned}\tag{9}$$

with homogeneous Dirichlet boundary condition.

Example 1. KPP-Fisher equation

- The KPP-Fisher equation is a nonlinear reaction-diffusion equation that models the propagation of a population.

$$\begin{aligned}\frac{\partial u}{\partial t} - \frac{\partial^2 u}{\partial x^2} &= u(1 - u), \quad t \in (0, T], \\ u(x, 0) &= u_0(x), \quad x \in (0, 1),\end{aligned}\tag{9}$$

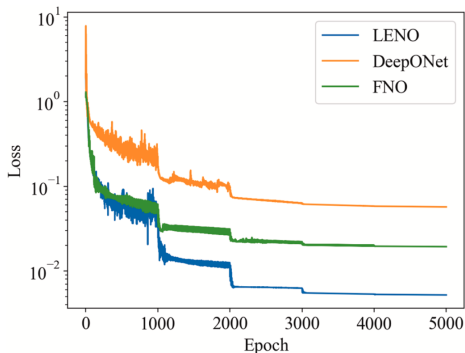
with homogeneous Dirichlet boundary condition.

- Data generation:
 - ▶ The initial condition u_0 is generated according to $u_0 \sim \mu = \mathcal{N}(0, 49(-\Delta + 7I)^{-2.5})$, where Δ is the Laplacian operator with homogeneous Dirichlet boundary condition and I is the identity.
 - ▶ The data is generated using a first order semi-implicit scheme with time step size $dt = 0.01$ for $T = 1.0$ and FEM with $N = 1024$ mesh points.

Results

Comparison of errors between **multilayer** LE-NO, DeepONet, and FNO on the 1D KPP-Fisher equation.

Method	E_{L^2}	E_{Res}	$E_{Nonlinear}$	Time/Epoch	#Parameter
LE-NO	5.54e-05	1.16e-03	1.17e-03	0.03s	1.13M
DeepONet	1.22e-04	6.20e-03	3.24e-03	0.50s	4.16M
FNO	9.31e-05	1.25e-03	1.26e-03	0.65s	2.12M



- Hao, W., & Wang, J. (2025). Laplacian eigenfunction-based neural operator for learning nonlinear partial differential equations. Journal of Computational Physics.

LENO: coupled system

- A coupled reaction-diffusion system in two variables:

$$\begin{cases} \partial_t u_1 - D_1 \Delta u_1 = \mathcal{F}_1(u_1, u_2), & \text{in } \Omega \times (0, T), \\ \partial_t u_2 - D_2 \Delta u_2 = \mathcal{F}_2(u_1, u_2), & \text{in } \Omega \times (0, T), \\ u_1(x, 0) = u_{1,0}(x), \quad u_2(x, 0) = u_{2,0}(x), & \text{in } \Omega, \end{cases} \quad (10)$$

where $u_1, u_2 : \Omega \rightarrow \mathbb{R}$ and $\mathcal{F}_1, \mathcal{F}_2$ are unknown nonlinear reaction terms.

LENO: coupled system

- A coupled reaction-diffusion system in two variables:

$$\begin{cases} \partial_t u_1 - D_1 \Delta u_1 = \mathcal{F}_1(u_1, u_2), & \text{in } \Omega \times (0, T), \\ \partial_t u_2 - D_2 \Delta u_2 = \mathcal{F}_2(u_1, u_2), & \text{in } \Omega \times (0, T), \\ u_1(x, 0) = u_{1,0}(x), \quad u_2(x, 0) = u_{2,0}(x), & \text{in } \Omega, \end{cases} \quad (10)$$

where $u_1, u_2 : \Omega \rightarrow \mathbb{R}$ and $\mathcal{F}_1, \mathcal{F}_2$ are unknown nonlinear reaction terms.

- LENO approximates each reaction term on the shared eigenbasis $\{\phi_i\}_{i=1}^P$:

$$\begin{aligned} \mathcal{F}_1(u_1, u_2) &\approx \mathcal{N}_1(u_1, u_2; \theta_1) = \sum_{i=1}^P \mathcal{G}_i(\beta(u_1), \beta(u_2); \theta_1) \phi_i(x), \\ \mathcal{F}_2(u_1, u_2) &\approx \mathcal{N}_2(u_1, u_2; \theta_2) = \sum_{i=1}^P \mathcal{G}_i(\beta(u_1), \beta(u_2); \theta_2) \phi_i(x), \end{aligned} \quad (11)$$

Reduced system for the coupled equations

- Galerkin approximation on $V_P = \text{span}\{\phi_i\}_{i=1}^P$:

$$\tilde{u}_k^n(x) = \sum_{i=1}^P \tilde{\beta}_{k,i}^n \phi_i(x), \quad k = 1, 2.$$

- Project the equations onto V_P and apply a semi-implicit scheme:

$$\begin{aligned} \frac{\tilde{\beta}_1^n - \tilde{\beta}_1^{n-1}}{t_n - t_{n-1}} + D_1 \Lambda_P \tilde{\beta}_1^n &= (\mathcal{F}_1(\tilde{u}_1^{n-1}, \tilde{u}_2^{n-1}), \Phi), \\ \frac{\tilde{\beta}_2^n - \tilde{\beta}_2^{n-1}}{t_n - t_{n-1}} + D_2 \Lambda_P \tilde{\beta}_2^n &= (\mathcal{F}_2(\tilde{u}_1^{n-1}, \tilde{u}_2^{n-1}), \Phi), \end{aligned} \quad (12)$$

where $\Lambda_P = \text{diag}(\lambda_1, \dots, \lambda_P)$ and $(f, \Phi)_i := (f, \phi_i)_{L^2}$.

- With LENO, replace the reaction terms by branch networks:

$$\begin{aligned} \frac{\tilde{\beta}_1^n - \tilde{\beta}_1^{n-1}}{t_n - t_{n-1}} + D_1 \Lambda_P \tilde{\beta}_1^n &= \mathcal{G}^{(1)}(\tilde{\beta}_1^{n-1}, \tilde{\beta}_2^{n-1}; \theta_1), \\ \frac{\tilde{\beta}_2^n - \tilde{\beta}_2^{n-1}}{t_n - t_{n-1}} + D_2 \Lambda_P \tilde{\beta}_2^n &= \mathcal{G}^{(2)}(\tilde{\beta}_1^{n-1}, \tilde{\beta}_2^{n-1}; \theta_2), \end{aligned} \quad (13)$$

Example 2. Gray-Scott model

- The Gray-Scott model is given by

$$\left\{ \begin{array}{ll} A_t - D_A \Delta A = SA^2 - (\mu + \rho)A, & x \in (0, 2\pi)^d, \ t \in (0, T), \\ S_t - D_S \Delta S = -SA^2 + \rho(1 - S), & x \in (0, 2\pi)^d, \ t \in (0, T), \\ A(x, 0) = A_0(x), & x \in (0, 2\pi)^d, \\ S(x, 0) = S_0(x), & x \in (0, 2\pi)^d. \end{array} \right. \quad (14)$$

- The initial condition A_0 and S_0 are generated according to $A_0 \sim \mu = \mathcal{N}(0, 49(-\Delta + 7I)^{-2.5})$ and $S_0 \sim \mu = \mathcal{N}(0, 49(-\Delta + 7I)^{-2.5})$, where Δ is the Laplacian operator with Neumann boundary condition and I is the identity operator.
- In 1D, the data is generated using a first order semi-implicit scheme with time step size $dt = 0.01$ for $T = 1$ and FEM with $N = 1024$ mesh points.

References

Network architecture and applications:

- Hao, W., & Wang, J. (2025). Laplacian eigenfunction-based neural operator for learning nonlinear partial differential equations. *Journal of Computational Physics*.
- Lu, L., Jin, P., Pang, G., Zhang, Z., & Karniadakis, G. E. (2021). Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature machine intelligence*, 3(3), 218-229.
- Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., & Anandkumar, A. (2021). Fourier neural operator for parametric partial differential equations. *ICLR*.
- Xu, X., Dan, T., Zhou, Z., Li, B., Wu, G., & Hao, W. (2026, In preparation). Neural operator-based digital twins predict personalized $A\beta$ -Tau propagation and optimize treatment in Alzheimer's disease.

References

Network architecture and applications:

- Hao, W., & Wang, J. (2025). Laplacian eigenfunction-based neural operator for learning nonlinear partial differential equations. *Journal of Computational Physics*.
- Lu, L., Jin, P., Pang, G., Zhang, Z., & Karniadakis, G. E. (2021). Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature machine intelligence*, 3(3), 218-229.
- Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., & Anandkumar, A. (2021). Fourier neural operator for parametric partial differential equations. *ICLR*.
- Xu, X., Dan, T., Zhou, Z., Li, B., Wu, G., & Hao, W. (2026, In preparation). Neural operator-based digital twins predict personalized $A\beta$ -Tau propagation and optimize treatment in Alzheimer's disease.

Universal approximation theorem:

- Chen, T., & Chen, H. (1995). Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. *IEEE transactions on neural networks*, 6(4), 911-917.
- Kovachki, N., Li, Z., Liu, B., Azizzadenesheli, K., Bhattacharya, K., Stuart, A., & Anandkumar, A. (2023). Neural operator: Learning maps between function spaces with applications to pdes. *Journal of Machine Learning Research*, 24(89), 1-97.
- Kovachki, N., Lanthaler, S., & Mishra, S. (2021). On universal approximation and error bounds for Fourier neural operators. *Journal of Machine Learning Research*, 22(290), 1-76.

References

Network architecture and applications:

- Hao, W., & Wang, J. (2025). Laplacian eigenfunction-based neural operator for learning nonlinear partial differential equations. *Journal of Computational Physics*.
- Lu, L., Jin, P., Pang, G., Zhang, Z., & Karniadakis, G. E. (2021). Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature machine intelligence*, 3(3), 218-229.
- Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., & Anandkumar, A. (2021). Fourier neural operator for parametric partial differential equations. *ICLR*.
- Xu, X., Dan, T., Zhou, Z., Li, B., Wu, G., & Hao, W. (2026, In preparation). Neural operator-based digital twins predict personalized $A\beta$ -Tau propagation and optimize treatment in Alzheimer's disease.

Universal approximation theorem:

- Chen, T., & Chen, H. (1995). Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. *IEEE transactions on neural networks*, 6(4), 911-917.
- Kovachki, N., Li, Z., Liu, B., Azizzadenesheli, K., Bhattacharya, K., Stuart, A., & Anandkumar, A. (2023). Neural operator: Learning maps between function spaces with applications to pdes. *Journal of Machine Learning Research*, 24(89), 1-97.
- Kovachki, N., Lanthaler, S., & Mishra, S. (2021). On universal approximation and error bounds for Fourier neural operators. *Journal of Machine Learning Research*, 22(290), 1-76.

Code is available at [https:](https://github.com/georgexxu/Data-driven-Model-Discovery-Neural-Operators)

[//github.com/georgexxu/Data-driven-Model-Discovery-Neural-Operators](https://github.com/georgexxu/Data-driven-Model-Discovery-Neural-Operators).